

AEGISPAY

Agentic Commerce Architecture V4

The protocol-aware Trust & Growth Layer for Agentic Commerce. AI proposes. AegisPay deterministically validates, authorizes and controls. Razorpay executes only approved actions.

Different protocols can enter AegisPay, but none can bypass the AegisPay Control Plane.

1. The Protocol Gateway

Every external protocol enters through **one** gateway, is normalized into a single canonical intent, and only then reaches the control plane. A new protocol is a new adapter — it changes the transport, never the money path.

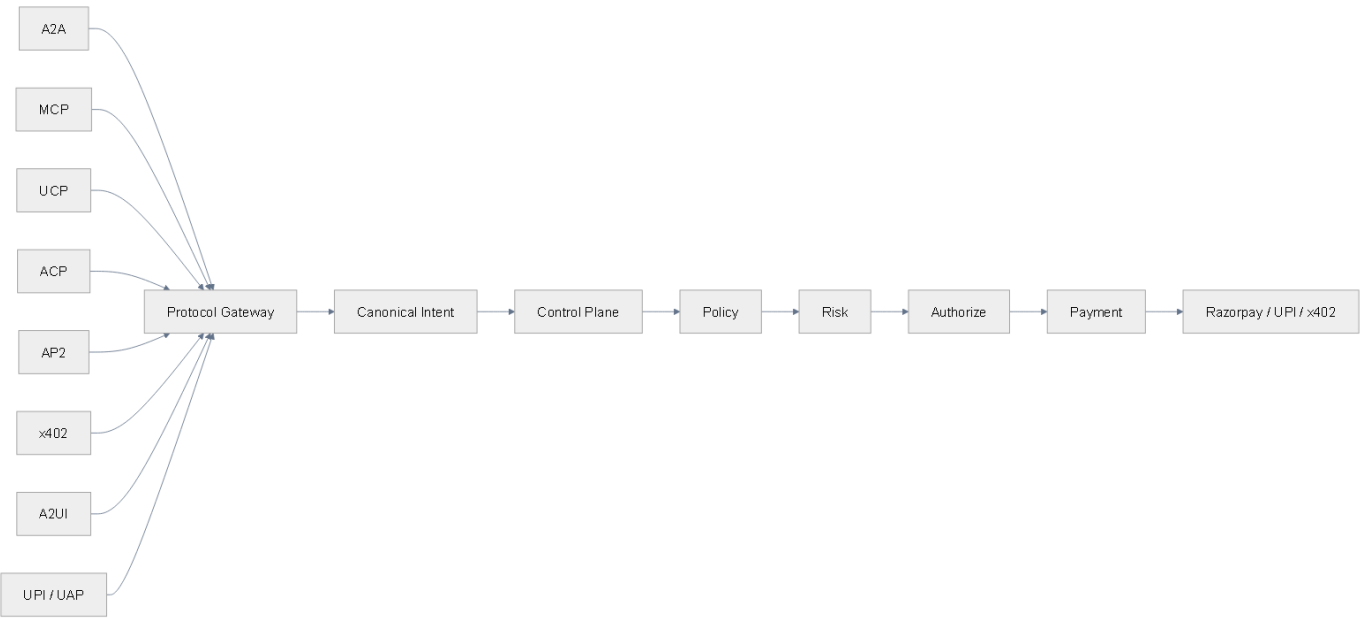


Figure 1 — All protocols collapse into one normalized intent

Protocols & maturity

Protocol	Role	Maturity
MCP	controlled agent tools / context	Core
A2A	agent-to-agent communication	Adapter-ready
UCP	commerce interoperability	Adapter-ready
ACP	agentic checkout / commerce	Adapter-ready
AP2	payment mandates & verifiable authorization	Experimental
x402	machine / pay-per-use payments	Experimental
A2UI	agent-driven UI	Experimental
NPCI UAP / UPI	India agentic-payment ecosystem	Future

Maturity is honest: **Core** is implemented, **Adapter-ready** has a mapping, **Experimental** maps only where semantics align, **Future** is a watch-list with no compliance claim.

2. The full system

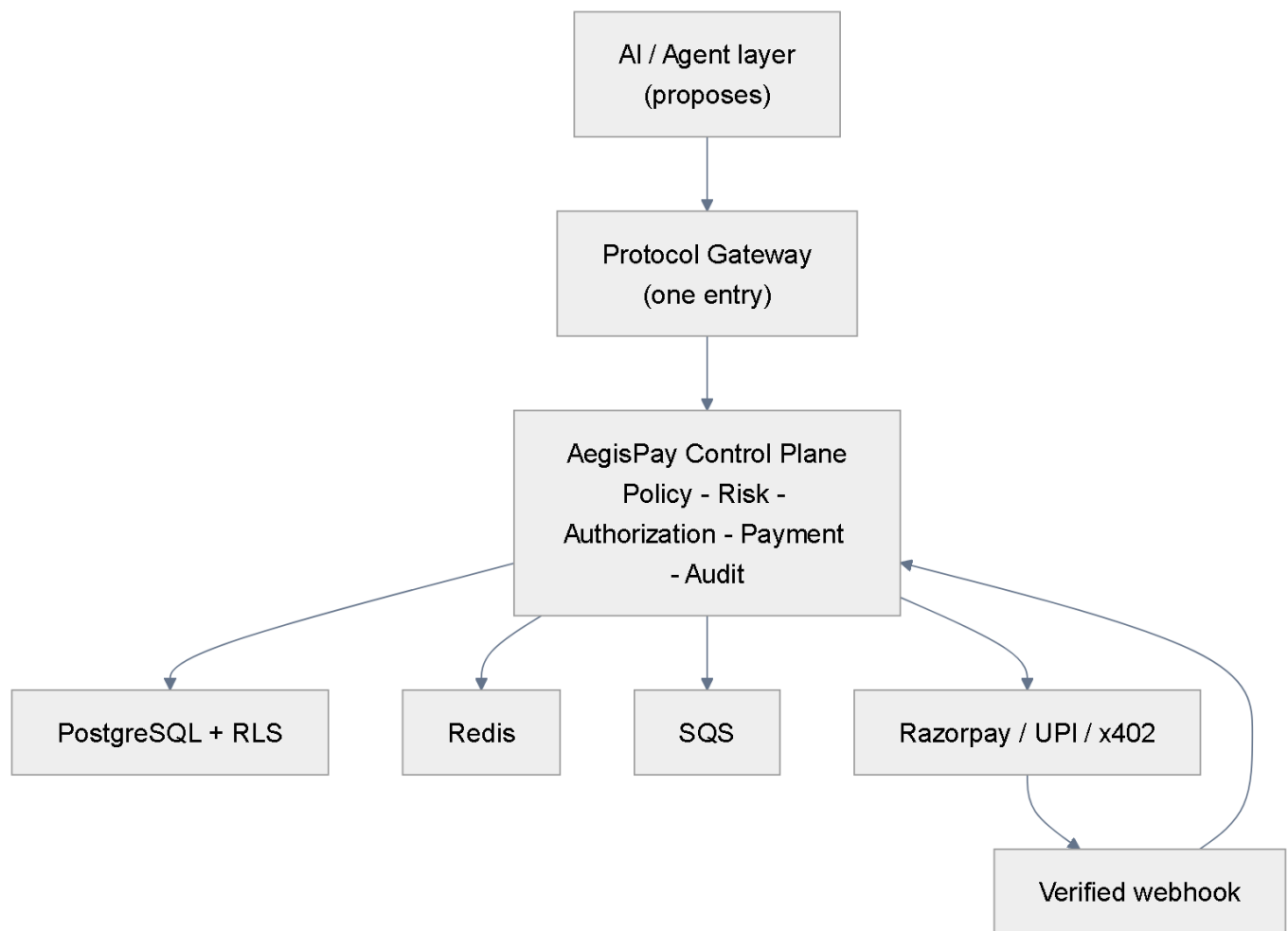


Figure 2 — One control plane, one money path

The AI layer proposes. Everything funnels through the gateway into the control plane, which gates and executes. There is exactly one money path.

AI / Control-plane boundary

- The AI runtime has **no payment keys, no database credentials, no unrestricted money tools, no policy mutation**.
- It can only produce an intent that the control plane gates — regardless of which protocol it arrived through.

3. Decision path (Policy · Risk · Authorization)

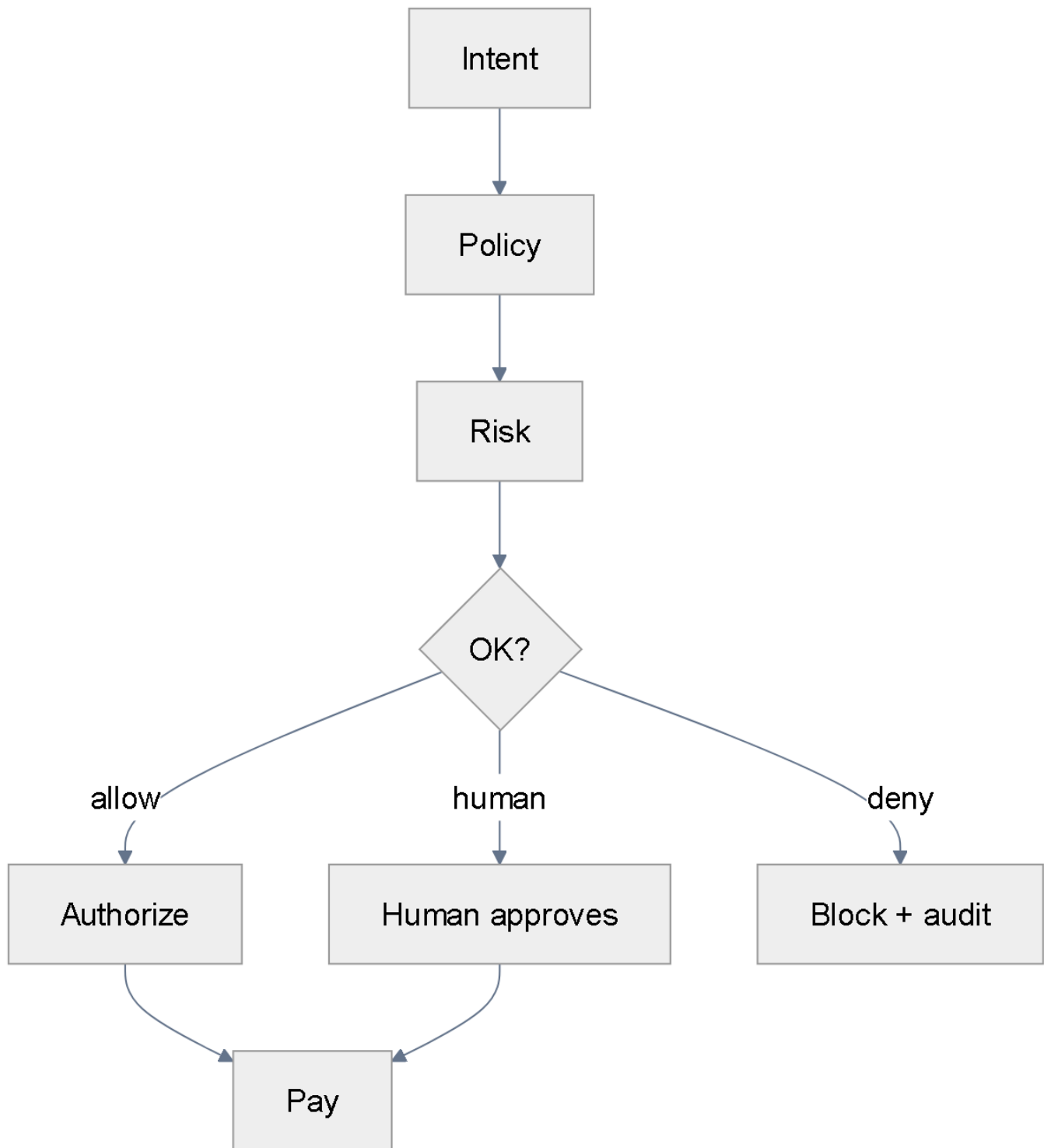


Figure 3 — Every money action passes a deterministic gate

- **Policy** is versioned and merchant-owned; the AI cannot change it.
- **Risk** is explainable: a score plus the reasons.
- **Authorization** is scoped, transaction-bound, expiring and single-use.
- **Human approval** happens only when required, then resumes from a checkpoint.

4. Payment state machine

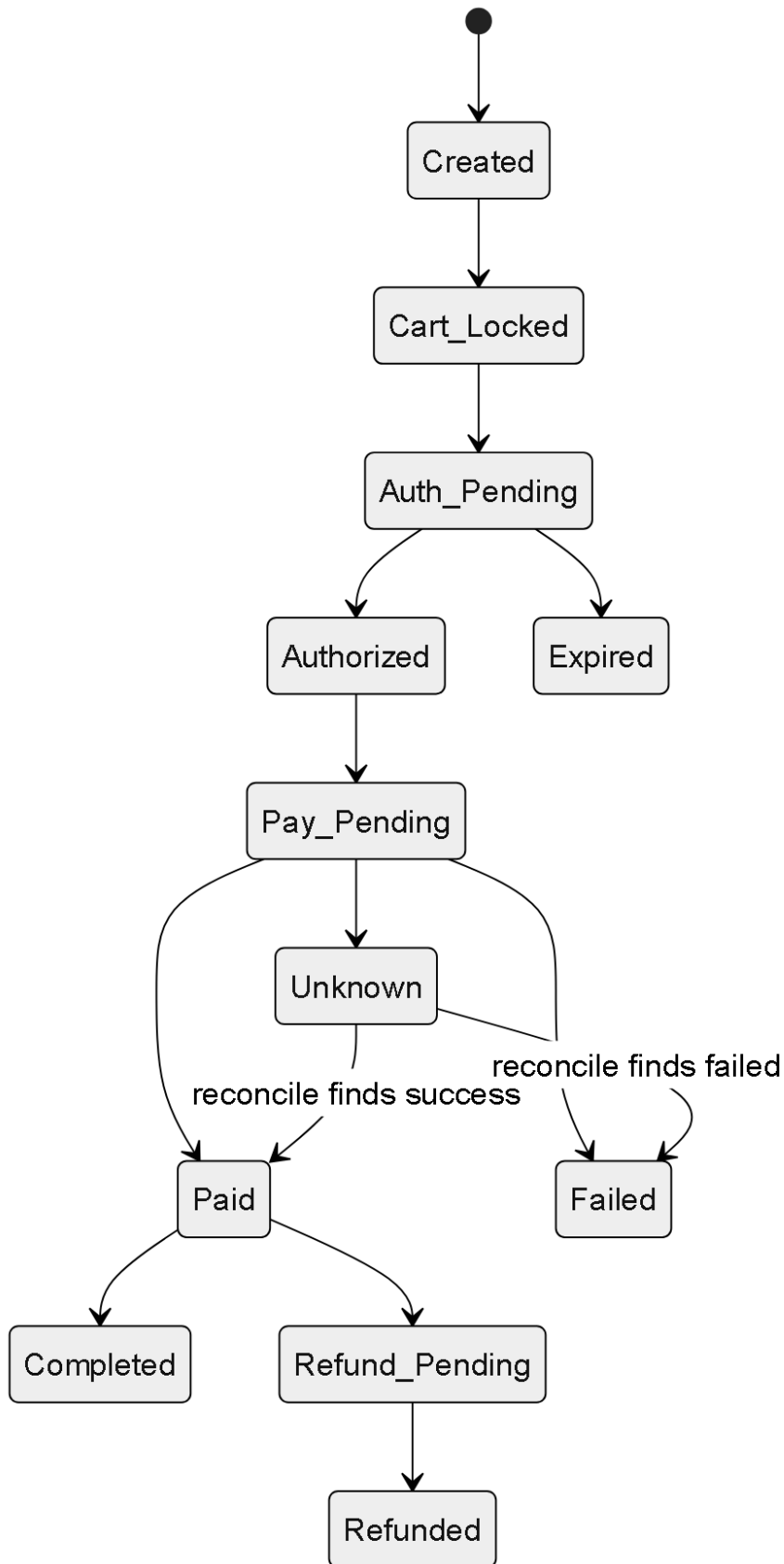


Figure 4 — UNKNOWN is first-class and resolved only by provider truth

We never blindly retry an UNKNOWN payment — we reconcile (ask the provider), then complete or fail safely.

5. Webhooks, idempotency & outbox

Concern	How it works
Webhook security	verify signature + timestamp, then deduplicate; bad/duplicate events are safe no-ops
Idempotency	every financial command carries a key; designed to prevent duplicate financial effects
Transactional outbox	state change + outbox event commit together; a worker publishes to SQS
Reconciliation	UNKNOWN payments are resolved from provider truth, never a blind retry

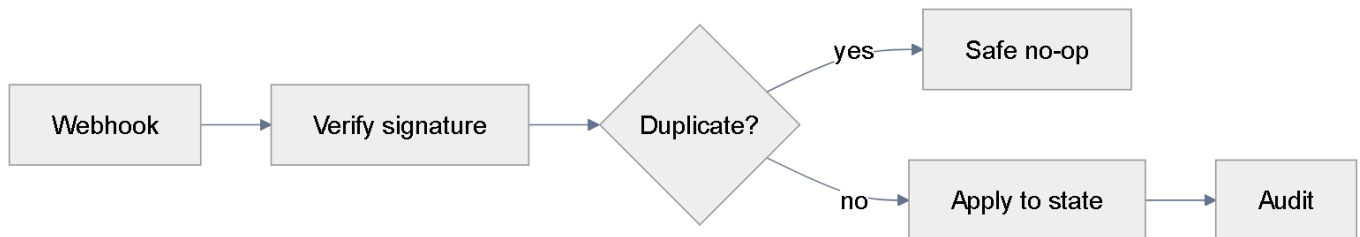


Figure 5 — Webhooks are untrusted until verified and deduped

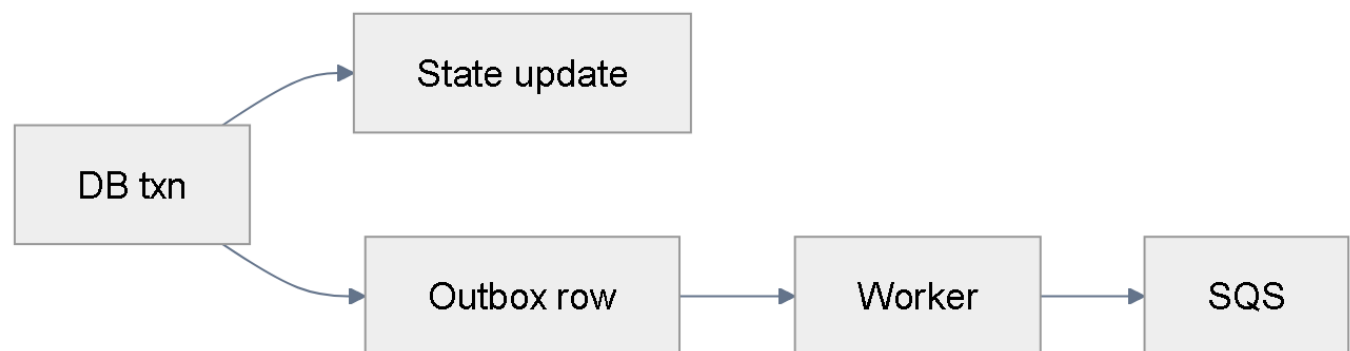


Figure 6 — State change and event are written in one transaction

6. SELL & GROW flows

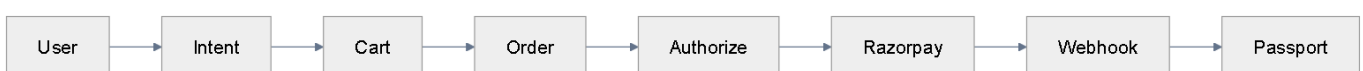


Figure 7 — SELL: intent → cart → order → authorize → Razorpay → webhook → Passport



Figure 8 — GROW: opportunity → campaign → budget ledger → orders → measure

7. Security & multi-tenancy

Concern	How it's handled
Protocol security	auth per protocol, schema validation, replay protection, tool allowlisting
Agent security	prompt-injection defense, no self-escalation, signed credentials, revocable
Cart / price / inventory	server-owned price, cart hash, price version, inventory reservation, expiry, product ownership lock
Refunds	controlled; the AI has no unrestricted refund tool
Multi-tenancy	shared PostgreSQL + RLS; app role cannot bypass RLS; tenant set server-side with SET LOCAL

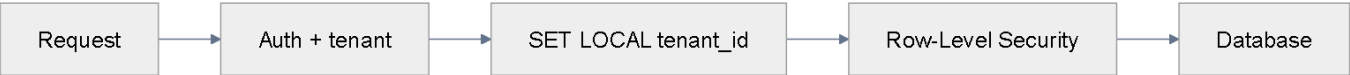


Figure 9 — Tenant context is set server-side; RLS enforces isolation

8. Production checklist

Area	In place
AI	no money path, tool allowlist, structured output, policy + risk, human approval
Protocols	single gateway, adapters normalize, protocol security, honest maturity
Payments	idempotency, state machine, webhook verify + dedupe, reconciliation, refund caps
Trust	versioned policy, explainable risk, scoped/expiring authz, Passport, outbox
Growth	atomic budget ledger, kill switch, A/B incremental measurement
Security	secrets isolation, RLS app role, encryption, rate limits
Reliability	timeouts, retries, circuit breakers, outbox + DLQ, backups + DR, SLOs

Why AegisPay is different

Different protocols can enter AegisPay, but none can bypass the AegisPay Control Plane.

- **One control plane.** Every protocol resolves to a normalized intent the control plane gates.
- **No direct money path.** The AI has no keys, no DB credentials, no money tools.
- **Provider truth.** Payment finality comes only from verified webhooks or reconciliation.
- **Deterministic authorization.** Policy, risk and a scoped/expiring authorization gate every action.
- **Tamper-evident.** Every decision is recorded in a hash-chained audit ledger and Transaction Passport.